

# 3653. XOR After Range Multiplication Queries I

Medium

Topics

Companies

Hint

You are given an integer array `nums` of length `n` and a 2D integer array `queries` of size `q`, where `queries[i] = [li, ri, ki, vi]`.

For each query, you must apply the following operations in order:

- Set `idx = li`.
- While `idx <= ri`:
  - Update: `nums[idx] = (nums[idx] * vi) % (109 + 7)`
  - Set `idx += ki`.

Return the **bitwise XOR** of all elements in `nums` after processing all queries.

## Example 1:

**Input:** `nums = [1,1,1]`, `queries = [[0,2,1,4]]`

**Output:** 4

### Explanation:

- A single query `[0, 2, 1, 4]` multiplies every element from index 0 through index 2 by 4.
- The array changes from `[1, 1, 1]` to `[4, 4, 4]`.
- The XOR of all elements is `4 ^ 4 ^ 4 = 4`.

## Example 2:

**Input:** `nums = [2,3,1,5,4]`, `queries = [[1,4,2,3],[0,2,1,2]]`

**Output:** 31

### Explanation:

- The first query `[1, 4, 2, 3]` multiplies the elements at indices 1 and 3 by 3, transforming the array to `[2, 9, 1, 15, 4]`.
- The second query `[0, 2, 1, 2]` multiplies the elements at indices 0, 1, and 2 by 2, resulting in `[4, 18, 2, 15, 4]`.
- Finally, the XOR of all elements is `4 ^ 18 ^ 2 ^ 15 ^ 4 = 31`.

### Constraints:

- $1 \leq n == \text{nums.length} \leq 10^3$
- $1 \leq \text{nums}[i] \leq 10^9$
- $1 \leq q == \text{queries.length} \leq 10^3$
- $\text{queries}[i] = [l_i, r_i, k_i, v_i]$
- $0 \leq l_i \leq r_i < n$
- $1 \leq k_i \leq n$
- $1 \leq v_i \leq 10^5$

### Python:

class Solution:

```
def xorAfterQueries(self, nums, queries):
```

```
    mod = 1000000007
```

```
    # Process each query
```

```
    for t in queries:
```

```
        l = t[0]
```

```
        r = t[1]
```

```
        k = t[2]
```

```
        v = t[3]
```

```
        idx = l
```

```
        # Apply operation at step k
```

```
        while idx <= r:
```

```
            temp = nums[idx]
```

```
            nums[idx] = (temp * v) % mod
```

```
            idx += k
```

```
    # Compute XOR of final array
```

```
    ans = 0
```

```
    for num in nums:
```

```
        ans ^= num
```

```
    return ans
```

## JavaScript:

```
var xorAfterQueries = function(nums, queries) {
    const MOD = 1e9 + 7;

    for (const [l, r, k, v] of queries) {
        for (let idx = l; idx <= r; idx += k) {
            nums[idx] = (nums[idx] * v) % MOD;
        }
    }

    let res = 0;
    for (const num of nums) {
        res ^= num;
    }

    return res;
};
```

## Java:

```
class Solution {
    final int mod = 1000000007;

    public int xorAfterQueries(int[] nums, int[][] queries) {

        // Process each query
        for (int[] t : queries) {
            int l = t[0];
            int r = t[1];
            int k = t[2];
            int v = t[3];

            int idx = l;

            // Apply operation at step k
            while (idx <= r) {
                long temp = nums[idx];
                nums[idx] = (int)((temp * v) % mod);
                idx += k;
            }
        }

        // Compute XOR of final array
        int ans = 0;
        for (int num : nums) {
```

```
        ans ^= num;
    }

    return ans;
}
```